

1. Project Title: AI-Powered Environmental Intelligence System for Air Quality Prediction and Smart Mobility Recommendations

2. Project Statement and Outcomes:

Air pollution poses serious health risks, yet most citizens lack access to real-time predictive air quality insights, particularly for their specific travel routes and planned locations. This project builds a full-stack AI-powered web application that integrates the user's live location data from their mobile device, historical and real-time AQI data sources, weather patterns, and traffic feeds to predict localized Air Quality Index values. The system generates personalized health advisories based on the user's current location and upcoming travel plans — alerting them proactively if their route passes through high-pollution zones during the predicted exposure window.

The deliverable is a location-aware air quality intelligence platform where users register health profiles, share location access, and input travel plans to receive route-specific AQI forecasts and real-time push notifications. The platform features an interactive pollution map, time-based AQI trend charts, travel route advisory overlays, and push notification delivery for critical pollution events — enabling citizens, commuters, and health-sensitive individuals to make informed decisions about outdoor activities and travel routes.

3. Modules to be Implemented:

1. Location & AQI Real-Time Data Integration
2. Predictive AQI Forecasting Model
3. Route Advisory, Dashboard & Push Notifications
4. System Integration, Testing & Project Finalization

4. Week-wise Module Implementation and High-Level Requirements:

Weeks 1–2: Location & AQI Real-Time Data Integration

- Integrate GPS/location APIs (browser Geolocation API, Google Maps API) for real-time user location capture with user consent workflows.
- Connect to AQI data sources (OpenAQ API, WAQI API, CPCB data feeds) for real-time and historical pollutant readings (PM2.5, PM10, NO2, O3, CO).
- Build a data ingestion pipeline that aggregates weather data (OpenWeatherMap) and traffic density data to correlate with pollution levels.
- Design user profile management with health sensitivity settings (asthmatic, elderly, children) that personalize alert thresholds.
- Set up a time-series database (InfluxDB) for storing sensor readings and user location history for personalized model training.

Weeks 3–4: Predictive AQI Forecasting Model

- Train an LSTM/GRU time-series model on historical AQI data combined with weather and traffic variables for 1–24 hour ahead AQI predictions.
- Implement spatial interpolation (Kriging/IDW) to estimate AQI at arbitrary user locations between monitoring stations.
- Build a travel route pollution estimator that calculates expected cumulative AQI exposure for planned routes using Google Directions API.
- Develop a FastAPI prediction service returning predicted AQI values, pollutant breakdowns, health impact classifications, and confidence intervals.
- Implement model retraining pipelines to update the forecasting model weekly with new sensor data batches.

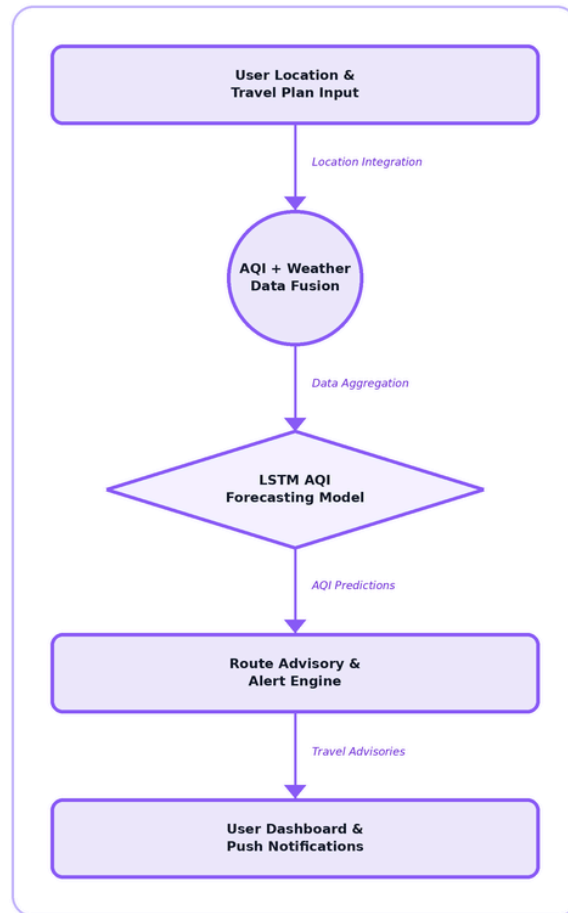
Weeks 5–6: Route Advisory, Dashboard & Push Notifications

- Build a travel plan input module where users enter source, destination, and travel time; the system generates a predicted AQI exposure profile for the route.
- Implement an alternative route suggestion feature proposing lower-pollution options based on real-time and predicted AQI maps.
- Develop a Firebase Cloud Messaging push notification engine for AQI threshold breach alerts and proactive travel advisories.
- Build a React.js/PWA dashboard with an interactive Leaflet.js pollution map, AQI trend charts, and travel advisory overlays.
- Implement a personal exposure history feature and notification preference management; deploy as a PWA for cross-platform mobile compatibility.

Weeks 7–8: System Integration, Testing & Project Finalization

- Conduct end-to-end integration testing across all modules — location capture → AQI forecaster → route advisory engine → user dashboard — and resolve any cross-module compatibility issues and edge-case failures.
- Perform load and stress testing on the backend APIs and cross-browser UI validation; address performance bottlenecks and implement enhancements from peer and mentor review.
- Refine the user experience based on testing feedback: improve interface clarity, add contextual help tooltips, and ensure full responsiveness on mobile and tablet devices.
- Prepare comprehensive project documentation covering system architecture, data flow diagrams, API reference, model or algorithm evaluation report, deployment guide, and known limitations.
- Develop the final project presentation (PPT) covering problem statement, proposed solution, architecture, key features, technology stack, results, challenges, and future scope — and rehearse the full end-to-end demo for a smooth, confident delivery.

5. Flow Chart:



6. Sample Dashboard:

